Lab Handout 4-H

Multi-Task Real-Time Inference, Latency Budget, and Final Model Selection

Capstone — Module 4: AI-Driven Classification Pipeline

1 Objective

Assemble the end-to-end inference pipeline: build a reusable `InferenceEngine` that loads the trained classifier per task, run a latency microbenchmark, score the four model families against the deliverable rubric to pick the deployed model per task, and stream live predictions from the FPGA. Every score in the rubric is read mechanically from the LH4-A through LH4-G manifest files.

2 Prerequisites

- Lab Handouts 4-A through 4-G complete and their manifests present in `Code/AIML/`: `validation_manifest.js` `preprocess_manifest.json`, `baselines_manifest.json`, `svm_manifest.json`, `mlp_manifest.json`, `cnn_manifest.json`, `stratified_manifest.json`.
- Per-task artifacts present in `Code/AIML/artifacts/<task>/`: `preprocessor.pkl`, `rf.pkl`, `svm.pkl`, `mlp.pt`, `cnn.pt`.
- LH3-F `collect.py` still functional; hardware producing feature vectors on demand if the live demo (Step 6) is exercised.

Tasks scoped in this handout

The four classification tasks are SF, modulation, emitter ID, and packet type. The `mod` task is auto-skipped throughout because only LoRa was collected in Module 3 (single class). The deployed pipeline therefore selects models for three tasks: `sf`, `beacon`, `pkt`.

3 Latency Budget

Stage	Budget (ms)	Source
RF on-air (SF7, 32 B)	∼50	LH3-D
Receiver firmware	<0.5	LH3-D latency budget
UART transport (43 B @ 115200)	3.73	LH3-D fixed calculation
FPGA pipeline	<0.1	Module 2 feature export loop
Python inference (this handout, 3 tasks)	**<8**	**target**
Total (end-to-end)	**<62**	well inside inter-packet interval

Table 1: Latency budget. The classifier has ∼996 ms of headroom between consecutive 1 s-interval ping-pong cycles, so an 8 ms inference ceiling is extremely conservative; hitting it leaves room for all three tasks to run in parallel without queueing.

4 Procedure

4.1 Step 1 — The `InferenceEngine` library

`Code/AIML/inference.py` is a library (no top-level work). It accepts a dict of task → chosen-model-name, loads the per-task preprocessor and the chosen model, and exposes `predict(feature_dict, seq_buffer)` that returns predictions for every loaded task. Each task uses its own scaler and its own post-drop feature column subset, so the caller passes *all* raw features as a dict and the engine picks what each task needs.

```python
import joblib
import numpy as np
import torch
import torch.nn as nn
from pathlib import Path


# Inlined to avoid re-running training scripts on import.
class MLP(nn.Module):
    def __init__(self, n_features, n_classes, hidden=(64, 32), p=0.3):
        super().__init__()
        layers = []; prev = n_features
        for h in hidden:
            layers += [nn.Linear(prev, h), nn.ReLU(), nn.Dropout(p)]
            prev = h
        layers.append(nn.Linear(prev, n_classes))
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x)


```

```python
class CNN1D(nn.Module):
    def __init__(self, n_features, n_classes):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv1d(n_features, 16, kernel_size=3, padding=1), nn.ReLU(),
            nn.Conv1d(16,         32, kernel_size=3, padding=1), nn.ReLU(),
            nn.AdaptiveAvgPool1d(1),
            nn.Flatten(),
            nn.Linear(32, 32), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(32, n_classes),
        )

    def forward(self, x):
        return self.net(x.transpose(1, 2))


class InferenceEngine:
    """Load one trained model per task and run them all on each frame.

    `model_choice`: dict task -> 'rf'/'svm'/'mlp'/'cnn'. Tasks present
    in the dict are loaded; tasks absent are skipped. Single-class
    tasks (mod) typically aren't included.
    """
    def __init__(self, model_choice, artifacts_dir):
        self.artifacts_dir = Path(artifacts_dir)
        self.choice = dict(model_choice)
        self.models = {}; self.prep = {}
        for task, choice in self.choice.items():
            d = self.artifacts_dir / task
            self.prep[task] = joblib.load(d / "preprocessor.pkl")
            self.models[task] = self._load_one(task, choice, d)

    def _load_one(self, task, choice, d):
        if choice in ("rf", "svm"):
            return {"kind": "sk",
                    "model": joblib.load(d / f"{choice}.pkl")["model"]}
        if choice == "mlp":
            ck = torch.load(d / "mlp.pt", map_location="cpu",
                            weights_only=False)
            m = MLP(ck["n_features"], ck["n_classes"])
            m.load_state_dict(ck["state_dict"]); m.eval()
            return {"kind": "mlp", "model": m}
        if choice == "cnn":
            ck = torch.load(d / "cnn.pt", map_location="cpu",
                            weights_only=False)
            m = CNN1D(ck["n_features"], ck["n_classes"])
            m.load_state_dict(ck["state_dict"]); m.eval()
            return {"kind": "cnn", "model": m, "window": 8}
        raise ValueError(f"Unknown model choice: {choice}")

    def predict(self, feature_dict, sequence_buffer=None):
        """feature_dict: {column_name: float}. The engine subsets and
```

```python
        scales per task using each task's saved preprocessor.

        sequence_buffer: optional dict task -> deque of length 8 of
        already-scaled feature vectors (CNN tasks only). The caller
        is responsible for pushing each frame's scaled vector into
        the deque before calling predict.
        """
        out = {}
        for task, bundle in self.models.items():
            pre = self.prep[task]
            cols = pre["feature_cols"]
            try:
                row = np.asarray([feature_dict[c] for c in cols],
                                 dtype=np.float32).reshape(1, -1)
            except KeyError as e:
                raise KeyError(
                    f"Task {task} needs column {e} but feature_dict "
                    f"has only {sorted(feature_dict)}") from None
            x_scaled = pre["scaler"].transform(row)
            if bundle["kind"] == "sk":
                idx = int(bundle["model"].predict(x_scaled)[0])
                # Most sklearn classifiers expose predict_proba; SVC
                # only does if it was trained with probability=True
                # (LH4-D's model is not).
                prob = None
                try:
                    prob = float(bundle["model"].predict_proba(x_scaled).
                        max())
                except (AttributeError, NotImplementedError):
                    pass
                except Exception:
                    pass
            elif bundle["kind"] == "mlp":
                with torch.no_grad():
                    logits = bundle["model"](torch.from_numpy(x_scaled))
                    probs = torch.softmax(logits, dim=1).numpy()[0]
                    idx = int(probs.argmax()); prob = float(probs.max())
            elif bundle["kind"] == "cnn":
                if sequence_buffer is None or task not in sequence_buffer:
                    raise ValueError(f"CNN task {task} needs a "
                                     f"sequence_buffer entry")
                seq = np.stack(list(sequence_buffer[task]))
                seq_t = torch.from_numpy(seq[None, :, :]).float()
                with torch.no_grad():
                    logits = bundle["model"](seq_t)
                    probs = torch.softmax(logits, dim=1).numpy()[0]
                    idx = int(probs.argmax()); prob = float(probs.max())
            label = pre["encoder"].inverse_transform([idx])[0]
            out[task] = {"label": label, "prob": prob, "kind": bundle["
                kind"]}
        return out
```

Listing 1: `inference.py` — model-agnostic per-task predict, with inlined model classes.

4.2 Step 2 — Final model-selection rubric (auto-scored from manifests)

Score each of the four models per task on a 0–5 scale against each criterion. Accuracy and robustness are read from the LH4 manifests; latency comes from Step 3's microbenchmark; interpretability and complexity are fixed-per-family constants the script applies uniformly across tasks.

Criterion	Weight	Source of evidence
Accuracy (test set)	0.40	`baselines_manifest.json` (RF), `svm/mlp/cnn_manifest.json`
Robustness (worst-bucket SNR)	0.25	`stratified_manifest.json`
Latency (median, ms)	0.15	Step 3 microbenchmark
Interpretability	0.10	RF=5, SVM=4, MLP=2, CNN=2 (fixed)
Deployment complexity (lower=better)	0.10	RF/SVM=5, MLP=3, CNN=2 (fixed)

Table 2: Model selection rubric. The script normalizes accuracy/robustness/latency to 0–5 across the four candidates and applies the weights. Highest weighted score per task is the deployed model.

```python
import json
import logging
import sys
from datetime import datetime, timezone
from pathlib import Path

import numpy as np

SCRIPT_DIR = Path(__file__).parent

WEIGHTS = {"acc": 0.40, "rob": 0.25, "lat": 0.15,
           "interp": 0.10, "complex": 0.10}
INTERP_FIXED   = {"rf": 5, "svm": 4, "mlp": 2, "cnn": 2}
COMPLEX_FIXED  = {"rf": 5, "svm": 5, "mlp": 3, "cnn": 2}


def load_acc_per_task():
    rf  = json.loads((SCRIPT_DIR / "baselines_manifest.json").read_text())
    svm = json.loads((SCRIPT_DIR / "svm_manifest.json").read_text())
    mlp = json.loads((SCRIPT_DIR / "mlp_manifest.json").read_text())
    cnn = json.loads((SCRIPT_DIR / "cnn_manifest.json").read_text())
    out = {}
    for task in ("sf", "beacon", "pkt"):
        out[task] = {
            "rf":  (rf["results"][task].get("rf")  or {}).get("acc"),
            "svm": svm["results"][task].get("test_acc"),
            "mlp": mlp["results"][task].get("test_acc"),
            "cnn": cnn["results"][task].get("test_acc"),
        }
    return out


```

```python
def load_robustness_per_task():
    s = json.loads((SCRIPT_DIR / "stratified_manifest.json").read_text())
    return s.get("robustness", {})


def normalize_to_0_5(values):
    """Map a list of floats (or None) to 0..5 by min-max within the
    list. Missing values get the minimum score (0). If all values are
    equal, every model gets 5."""
    arr = np.asarray([v if v is not None else np.nan for v in values],
                     dtype=float)
    if np.all(np.isnan(arr)):
        return [0.0] * len(values)
    mn = np.nanmin(arr); mx = np.nanmax(arr)
    if mx == mn:
        return [5.0 if not np.isnan(v) else 0.0 for v in arr]
    out = []
    for v in arr:
        if np.isnan(v):
            out.append(0.0)
        else:
            out.append(5.0 * (v - mn) / (mx - mn))
    return out


def latency_inverse_to_0_5(values):
    """Lower latency = higher score. Same min-max but inverted."""
    arr = np.asarray([v if v is not None else np.inf for v in values],
                     dtype=float)
    finite = arr[np.isfinite(arr)]
    if len(finite) == 0:
        return [0.0] * len(values)
    mn = finite.min(); mx = finite.max()
    if mx == mn:
        return [5.0 if np.isfinite(v) else 0.0 for v in arr]
    return [5.0 * (mx - v) / (mx - mn) if np.isfinite(v) else 0.0
            for v in arr]


def score_per_task(accs, robs, lats):
    """accs: {task: {model: acc}}, robs: {task: {model: {min,...}}},
       lats: {model: median_ms}. Returns {task: {model: weighted_total}}."
          ""
    candidates = ["rf", "svm", "mlp", "cnn"]
    rows = {}
    for task, by_model in accs.items():
        acc_vec = [by_model.get(m) for m in candidates]
        rob_vec = [(robs.get(task, {}).get(m) or {}).get("min")
                   for m in candidates]
        lat_vec = [lats.get(m) for m in candidates]
        a5 = normalize_to_0_5(acc_vec)
        r5 = normalize_to_0_5(rob_vec)
        l5 = latency_inverse_to_0_5(lat_vec)
        scores = {}
```

```
        for i, m in enumerate(candidates):
            total = (WEIGHTS["acc"]      * a5[i]
                     + WEIGHTS["rob"]    * r5[i]
                     + WEIGHTS["lat"]    * l5[i]
                     + WEIGHTS["interp"]  * INTERP_FIXED[m]
                     + WEIGHTS["complex"] * COMPLEX_FIXED[m])
            scores[m] = {"total": total,
                         "components": {"acc": a5[i], "rob": r5[i],
                                        "lat": l5[i],
                                        "interp": INTERP_FIXED[m],
                                        "complex": COMPLEX_FIXED[m]}}
        rows[task] = scores
    return rows
```

Listing 2: `select_model.py` — reads every manifest and ranks candidates.

4.3 Step 3 — Latency microbenchmark

For each candidate model and each non-skipped task, measure 1000 warm predictions on a fake feature row (and a zero-padded sequence buffer for the CNN) and report median, mean, p95, p99, max in milliseconds. The median latency feeds the rubric in Step 2; the p99 is the deployment-relevant tail.

```
import time
from collections import deque
from inference import InferenceEngine
import numpy as np

def microbenchmark(model_kind, n_warmup=20, n_iter=1000):
    """Per-model latency: build an engine that uses model_kind for every
    non-skipped task, then time predict()."""
    tasks = ["sf", "beacon", "pkt"]
    eng = InferenceEngine({t: model_kind for t in tasks},
                          artifacts_dir=SCRIPT_DIR / "artifacts")
    # Build a zero-valued feature dict covering every column any task
        needs.
    cols = set()
    for pre in eng.prep.values():
        cols.update(pre["feature_cols"])
    feat = {c: 0.0 for c in cols}
    F = max(len(pre["feature_cols"]) for pre in eng.prep.values())
    seq_buf = {t: deque([np.zeros(len(eng.prep[t]["feature_cols"]),
                                  dtype=np.float32) for _ in range(8)],
                        maxlen=8)
               for t in eng.models}
    for _ in range(n_warmup):
        eng.predict(feat, seq_buf)
    lats = []
    for _ in range(n_iter):
        t0 = time.perf_counter()
        eng.predict(feat, seq_buf)
        lats.append((time.perf_counter() - t0) * 1000.0)
    lats = np.asarray(lats)
    return {"median": float(np.median(lats)),
```

```
            "mean":   float(lats.mean()),
            "p95":    float(np.percentile(lats, 95)),
            "p99":    float(np.percentile(lats, 99)),
            "max":    float(lats.max())}
```

Why p99, Not Just Mean?

Garbage collection, Python bytecode JIT warm-up, and occasional OS scheduling hiccups produce a long-tailed latency distribution. The mean can be comfortably under budget while the 99th percentile occasionally breaches it. If p99 > budget, you will see periodic late predictions in the live pipeline — a problem the mean would have hidden. The script reports both.

4.4 Step 4 — Driver: pick winning model per task and write the manifest

```
log.info("=== Latency microbenchmark (median ms) ===")
lats = {}
for m in ["rf", "svm", "mlp", "cnn"]:
    bench = microbenchmark(m)
    lats[m] = bench["median"]
    log.info(f"  {m:<5} median={bench['median']:.3f}  "
             f"p95={bench['p95']:.3f}  p99={bench['p99']:.3f}  "
             f"max={bench['max']:.3f}")

accs = load_acc_per_task()
robs = load_robustness_per_task()
log.info("=== Per-task scores (weighted, highest wins) ===")
rows = score_per_task(accs, robs, lats)
choices = {}
for task, scores in rows.items():
    log.info(f"--- {task} ---")
    ranked = sorted(scores.items(), key=lambda kv: -kv[1]["total"])
    for m, s in ranked:
        log.info(f"  {m:<5} total={s['total']:.3f}  "
                 f"acc={s['components']['acc']:.2f} "
                 f"rob={s['components']['rob']:.2f} "
                 f"lat={s['components']['lat']:.2f} "
                 f"interp={s['components']['interp']} "
                 f"complex={s['components']['complex']}")
    choices[task] = ranked[0][0]
    log.info(f"  WINNER: {ranked[0][0]} (score {ranked[0][1]['total']:.3f
        })")

manifest = {
    "generated_at": datetime.now(timezone.utc).isoformat(),
    "weights":      WEIGHTS,
    "latency_ms":   lats,
    "scores":       rows,
    "deployed_choice": choices,
}
(SCRIPT_DIR / "deployment_manifest.json").write_text(
    json.dumps(manifest, indent=2, default=str))
log.info("Wrote deployment_manifest.json")
```

4.5 Step 5 — Validation-manifest schema check

Before deploying, assert the live feature-vector schema (the columns `collect.py` emits) is a superset of every column any chosen model needs. The check fails closed: any missing column blocks deployment.

```
val_manifest = json.loads(
    (SCRIPT_DIR / "validation_manifest.json").read_text())
training_schema = set(val_manifest["schema"])

needed = set()
for task, choice in choices.items():
    pre = joblib.load(SCRIPT_DIR / "artifacts" / task / "preprocessor.pkl"
        )
    needed.update(pre["feature_cols"])

missing = needed - training_schema
if missing:
    log.error(f"Schema drift: deployed models need columns "
              f"{missing} not present in validation_manifest.")
    sys.exit(1)
log.info(f"Schema check PASS: all {len(needed)} needed columns "
         f"present in validation_manifest.")
```

4.6 Step 6 — Live streaming inference (hardware-side, optional)

A separate `Code/AIML/live_inference.py` script reuses the LH3-F frame-reader and the InferenceEngine to print live predictions. Run it against hardware with the cooperative-target ping-pong active and capture the output to `demo_log.txt` for the deliverable.

```
import time
from collections import deque
from pathlib import Path

import numpy as np
import serial

from inference import InferenceEngine

SCRIPT_DIR = Path(__file__).parent
import json
choices = json.loads((SCRIPT_DIR / "deployment_manifest.json").read_text()
    )
choices = choices["deployed_choice"]


def live_inference(port="COM3", baud=115200):
    eng = InferenceEngine(choices, artifacts_dir=SCRIPT_DIR / "artifacts")
    seq_buf = {t: deque([np.zeros(len(eng.prep[t]["feature_cols"]),
                                  dtype=np.float32) for _ in range(8)],
                        maxlen=8)
               for t in eng.models}
    with serial.Serial(port, baud, timeout=1.0) as ser:
        # Reuse the LH3-F collect.py helpers: read_frame() and
```

```python
        # parse_forwarding_frame() return a dict that lines up with
        # the column names InferenceEngine expects (rssi_dbm, snr_db,
        # inter_arrival_ms, ...).
        from collect import read_frame, parse_forwarding_frame
        last_rx_ts = None
        while True:
            buf = read_frame(ser)
            if buf is None:
                continue
            fv = parse_forwarding_frame(buf)
            ia = (fv["rx_timestamp_ms"] - last_rx_ts) if last_rx_ts else 0
            last_rx_ts = fv["rx_timestamp_ms"]
            feat = {**fv, "inter_arrival_ms": ia}
            # Push scaled vector into each task's CNN buffer (only used
            # if a task chose CNN, but it's cheap to maintain anyway).
            for t in eng.models:
                cols = eng.prep[t]["feature_cols"]
                row = np.asarray([feat[c] for c in cols], dtype=np.float32
                    )
                seq_buf[t].append(eng.prep[t]["scaler"].transform(
                    row.reshape(1, -1))[0])
            t0 = time.perf_counter()
            preds = eng.predict(feat, seq_buf)
            dt = (time.perf_counter() - t0) * 1000.0
            line = f"[{time.strftime('%H:%M:%S')}] "
            for t in ("sf", "beacon", "pkt"):
                p = preds.get(t)
                if p is None:
                    continue
                pstr = (f"p={p['prob']:.2f}" if p['prob'] is not None
                        else "p=?")
                line += f"{t.upper()}={p['label']} ({pstr}) | "
            line += f"lat={dt:.1f} ms"
            print(line)


if __name__ == "__main__":
    live_inference()
```

Listing 3: `live_inference.py` — minimal serial-streaming demo.

Module 4 Final Deliverable (LH4-H)

1. `Code/AIML/inference.py`: the `InferenceEngine` library.
2. `Code/AIML/select_model.py`: the runnable rubric driver that consumes every LH4 manifest and writes `deployment_manifest.json`.
3. `Code/AIML/live_inference.py`: the serial-streaming demo script (hardware-side).
4. `Code/AIML/deployment_manifest.json`: per-task component scores, weighted totals, latency stats (median/p95/p99), and the chosen model per non-skipped task.
5. Latency microbenchmark output (in the script's archived log): median, p95, p99, max for

each candidate model. The framework target is p99 < 8 ms; record the actual numbers and any exceedance with a one-sentence reason.

6. Filled model-selection rubric: a 4×3 table (four models $\times$ three non-skipped tasks) showing the weighted totals, with a short paragraph per task explaining the winning choice.
7. `demo_log.txt`: at least 5 minutes of live predictions from hardware (Step 6 only — skip if hardware not available, but note the skip in the retrospective), with ground-truth accuracy computed after the fact for the three deployed tasks.
8. `Code/AIML/Logs/select_model_<timestamp>.log`: archived audit trail for the rubric run.
9. One-page Module 4 retrospective: what the operating-condition shift revealed (LH4-F clean→contended), what the dataset-design alias revealed (LH4-C beacon/pkt structural ties), which feature additions or campaign collections would most likely improve the weakest task, and what the next campaign would look like.

5 Verification Checklist

- ☐ `InferenceEngine` loads each chosen model and returns predictions for a feature dict.
- ☐ Microbenchmark: latency stats logged for every candidate; p99 < 8 ms (or written justification if exceeded).
- ☐ Validation-manifest schema check passes (every column the deployed models need is in `validation_manifest.j`
- ☐ Rubric driver writes `deployment_manifest.json` with per-task weighted totals and the winning model.
- ☐ (Hardware-side, optional) Live inference runs uninterrupted for 5 minutes against hardware; `demo_log.txt` captured.
- ☐ One-page Module 4 retrospective complete.

Glossary

UART	Universal Asynchronous Receiver/Transmitter	2